MyOS

A 32-Bit Low-Resource Operating System with Composite GUI & JIT Forth Compiler

Phase 10 Custom BIOS Boot Stack & Graphics Engine Optimization Release

COMPREHENSIVE TECHNICAL MANUAL

Architecture Target:	Intel x86 (i686) 32-Bit Protected Mode (Flat Segmentation)
Custom Boot Stack:	Legacy BIOS Stage 1 MBR & Stage 2 Loaders (100% GRUB Independent)
Standard Boot Option:	Multiboot Compliant Header for GRUB2 / ISO Emulations
Graphics Interface:	VESA Bios Extensions (VBE 2.0) 1024x768x32bpp Linear Framebuffer
Console Engine:	High-Speed Decoupled Text Renderer with Double-Buffering
Built-in File System:	MyFS Flat Sector-Mapped Storage System
Hardware Controller:	ATA IDE Hard Disk Controller via PIO Ports (0x1F0-0x1F7)
Language Platform:	Console Shell CLI with an Integrated Native x86 JIT Forth Compiler
Hypervisor Support:	Oracle VirtualBox 7.x (Pre-configured UUID), VMware, QEMU
Release Specification:	Version 10.0-Stable (Optimized memory loops & zero-lag flushing)
Lead Engineer:	Shreearu Bisoi
Date of Publication:	May 22, 2026

1. Executive Overview & System Features

MyOS is a custom-built, bare-metal operating system designed from scratch for the Intel x86 (i686) 32-bit architecture. Rather than relying on heavy third-party code or GNU/Linux components, MyOS provides a highly optimized, original runtime environment implementing protected-mode memory structures, a Wayland-inspired graphical window manager, custom keyboard/mouse drivers, and a flat file storage engine. The system is designed to operate under extreme memory and CPU constraints while providing an interactive user interface and an embedded Just-In-Time (JIT) Forth compiler.

Core Operational Capabilities

- Hybrid Boot Stack: Supports both custom 16-bit legacy BIOS bootloaders (Stage 1 MBR & Stage 2 VBE initialization) and Multiboot-compliant GRUB2 boot configs.
- 32-Bit Flat Protected Mode: Configures the Global Descriptor Table (GDT) and Interrupt Descriptor Table (IDT) with a fully remapped 8259 PIC controller.
- Efficient Memory Allocation: Custom physical memory manager (PMM) driven by a page-aligned, single-bit bitmap frame allocator.
- Interactive Graphical Compositor: Sleek VESA double-buffered graphics engine displaying a custom desktop background, draggable windows, desktop taskbar, realtime system clock, and mouse cursor renderer.
- Built-in ATA IDE Driver: Operates directly via x86 I/O ports in PIO mode, with built-in SATA-bridge protections and command timeouts to prevent hypervisor hangs.
- MyFS File Storage: A direct sector-mapped flat file storage layer that mounts static assets, text screens, and Forth script definitions directly.
- Extensible Forth Interpreter & x86 JIT Compiler: An embedded interactive console engine featuring a JIT assembler that compiles Forth words into native machine code directly on the heap for bare-metal speeds.

2. Phase 10 Optimization & VirtualBox Debug

In Phase 10, MyOS underwent a detailed debugging and code audit phase to resolve boot failure issues on Oracle VirtualBox, while simultaneously rewriting critical rendering paths. These changes resolved virtual machine hangs and boosted overall text rendering performance up to 30x.

1. Resolving the Stage 2 Segment Pointer Mismatch

When booting MyOS via raw disk images (.img/.vdi) on legacy BIOS hypervisors, the custom Stage 1 MBR (boot1.asm) sequentially loads Stage 2 sectors. A subtle bug existed: each sector read loop incremented the Extra Segment (ES) register. By the time Stage 1 jumped to Stage 2 (boot2.asm) at address 0x8000, ES held a dirty value (typically 0x2000). When Stage 2 queried VESA BIOS Extensions (INT 0x10) to obtain VBE Mode Info, it wrote the structures to ES:DI = 0x2000:0x7000 (physical 0x27000). However, Stage 2 loaded parameters from DS:0x7000, which mapped to physical 0x07000. This resulted in uninitialized garbage memory being passed to the C kernel as the linear framebuffer base address, causing the CPU to immediately crash/lock on the first pixel draw, leading to a permanent black screen.

THE SEGMENT POINTER FIX (BOOT2.ASM)

We injected explicit segment resets at the absolute entry point of Stage 2 (boot2.asm):

```
stage2_entry:
    xor ax, ax
    mov es, ax
    mov ds, ax
```

This aligns ES and DS directly to 0x0000. VBE Mode Info queries write to 0x0000:0x7000 (physical 0x07000), allowing the kernel loader to read the precise framebuffer base pointer, resolving the VirtualBox black screen.

2. Early Framebuffer Console Redirection

Historically, console logging was redirected to the VESA framebuffer late in kernel initialization (kernel.c). If a CPU exception or physical memory error occurred early in kmain(), error logs defaulted to standard VGA text memory (0xB8000). Because VESA was already active, this memory was invisible, rendering early errors completely hidden behind a blank black screen. We moved fb_init() and shell_init() to the very first instructions of kmain(), so early logs display instantly on the screen.

3. Graphics Rendering & Memory Performance Tuning

A full-screen framebuffer redraw at 1024x768x32bpp requires copying 3,145,728 bytes of memory from the back buffer to the active screen. To ensure a smooth GUI and console shell without CPU spikes, we implemented two critical performance refactors.

1. High-Performance 32-Bit Double-Word Memory Loops

The standard memory copy and fill functions (memcpy and memset in src/string.c) originally worked byte-by-byte. Performing 3MB screen swaps byte-by-byte wasted hundreds of millions of cycles per second in virtualization. We optimized these functions to perform 32-bit aligned copies (double-words) using 4-byte loops. This provides a 4x to 8x hardware speedup for all standard memory buffer sweeps.

```
void* memcpy(void* dst, const void* src, size_t n) {
    uint8_t* d = (uint8_t*)dst;
    const uint8_t* s = (const uint8_t*)src;

    size_t n4 = n / 4;
    uint32_t* d4 = (uint32_t*)dst;
    const uint32_t* s4 = (const uint32_t*)src;
    for (size_t i = 0; i < n4; i++) {
        d4[i] = s4[i];
    }
    for (size_t i = n4 * 4; i < n; i++) {
        d[i] = s[i];
    }
    return dst;
}
```

2. Decoupled Buffer Swapping (Smart Console Flushing)

Previously, the console character function (con_putchar) automatically triggered a full-screen fb_swap() buffer copy every time a single letter was drawn. When printing a single 80-character line of text, the CPU was forced to copy 3MB of pixels 80 times consecutively (totaling 240MB of copies!), resulting in immense lag. We completely decoupled double-buffering from con_putchar and introduced a manual con_flush() handler. We now flush only when printing newlines, executing command strings, or reading key presses, reducing copy overhead up to 30x.

```
// In src/shell.c - Optimizing the rendering trigger
void con_flush(void) {
    if (use_fb) {
        fb_swap(); // Transfer back buffer to screen once at string completion
    }
}
```

4. Custom Legacy BIOS Boot Stack Architecture

A standout feature of MyOS is its custom 16-bit legacy BIOS bootloader stack. While GRUB is supported via Multiboot headers, our native bootloader stack allows the operating system to boot directly from a raw floppy or disk image without any third-party dependencies.

Stage 1 - Master Boot Record (src/boot1.asm)

The MBR is placed exactly in Sector 0 of the drive. The BIOS reads it into memory at 0x7C00. The Stage 1 loader performs basic environment normalization, registers the active boot drive index, and uses BIOS interrupt 13h to load Stage 2 sectors. Specifically, it reads 192 sectors (representing up to 96KB of program code) from the drive and loads them into memory address 0x8000.

Stage 2 - System Initializer & Kernel Loader (src/boot2.asm)

Stage 2 executes in 16-bit Real Mode at address 0x8000. It coordinates the hardware configuration and transitions to 32-bit Protected Mode. The steps execute in the following sequence:

- Normalize Registers: Clears segment registers ES and DS to 0 to prevent pointer offsets.
- VESA BIOS Extensions (VBE): Queries VBE controller capabilities (INT 0x10, AX=0x4F00) and specific mode information (INT 0x10, AX=0x4F01) for mode 0x118 (1024x768x32bpp). It enters graphics mode by executing INT 0x10, AX=0x4F02.
- Global Descriptor Table: Initializes a temporary GDT to map a flat physical 4GB memory space (Base 0x00000000, Limit 0xFFFFFFFF) with simple code and data access permissions.
- Protected Mode Transition: Disables interrupts (cli), enables the A20 gate, sets the Protection Enable bit in Control Register 0 (CR0 |= 0x1), and executes a far jump into the 32-bit code segment.
- Kernel Image Relocation: Reads the kernel raw sectors starting from Sector 2, copies the binary directly into physical RAM at the 1MB boundary (0x100000), and jumps directly to the entry point.

DIRECT HARD DRIVE LAYOUT
Sector 0: Stage 1 Boot MBR (512 Bytes, finishes with 0xAA55 signature)
Sector 1: Stage 2 Bootloader (512 Bytes)
Sectors 2+: Packed Flat Binaries (Kernel Code & Data, JIT Compiler, Static Assets)

5. Core Kernel Architecture & Hardware Drivers

The MyOS C kernel manages hardware configuration, interrupts, memory allocation, and basic device drivers in a unified 32-bit Protected Mode environment.

1. Global Descriptor Table (gdt.c)

The GDT defines the base address, limit, and privilege level of the segment registers. MyOS uses a flat segmentation model (5 GDT entries): a Null descriptor, a Kernel Code Segment, a Kernel Data Segment, a User Code Segment, and a User Data Segment. The base of all segments is 0x00000000 with a limit of 0xFFFFFFFF, giving full access to physical RAM.

2. Interrupt Dispatching & PIC Remapping (idt.c, isr.asm)

The Interrupt Descriptor Table contains 256 gates mapping CPU interrupts to interrupt service routines. By default, the Intel 8259 Programmable Interrupt Controller (PIC) maps hardware interrupts (IRQs) to interrupt vectors 0x08-0x0F, which conflict with CPU Exception vectors. We remap the PIC to vector offsets 0x20 (Master) and 0x28 (Slave). CPU exceptions (double faults, page faults) are caught via assembly stubs in isr.asm and routed to structured kernel panic dialogs.

3. Physical Memory Manager (pmm.c)

The Physical Memory Manager tracks used and free physical memory page frames (4KB blocks). It uses a flat bitmap located above the kernel space where each bit represents a page. A bit value of 1 signifies an allocated page, while 0 represents a free page. PMM functions include pmm_alloc() and pmm_free() which run quick bitmask operations to allocate sequential frames.

4. Hard Disk ATA Controller (ata.c)

MyOS includes a built-in ATA IDE hard drive driver that uses direct x86 I/O port instructions (ports 0x1F0 to 0x1F7 for Primary Master). It runs in Polled I/O (PIO) mode. To support slower SATA-to-IDE emulation layers in hypervisors like VMware, the read and write commands incorporate strict status timeouts and data-ready flags to prevent CPU freeze states during sector lookups.

```
// Direct port communication in src/ata.c
void ata_read_sectors(uint32_t lba, uint8_t count, uint16_t* buf) {
    outb(0x1F2, count);
    outb(0x1F3, (uint8_t)lba);
    outb(0x1F4, (uint8_t)(lba >> 8));
    outb(0x1F5, (uint8_t)(lba >> 16));
    outb(0x1F6, 0xE0 | ((lba >> 24) & 0x0F));
    outb(0x1F7, 0x20); // Send ATA Read command
    // Poll status register with timeout protection...
}
```

6. Wayland-Inspired Graphical Composite Desktop

MyOS implements a complete custom window compositor and desktop manager in graphics mode. Rather than drawing windows directly to physical video memory, windows are rendered to private back-buffers and assembled dynamically by the compositor, preventing tearing and rendering artifacts.

1. VESA Framebuffer Graphics (framebuffer.c)

The VESA LFB driver maps the physical video memory base address queried during boot. It supports high-resolution pixel mapping (1024x768 pixels with 32-bit true color, where each pixel consumes 4 bytes: Alpha, Red, Green, Blue). We implement full-page double-buffering by allocating a back buffer in system RAM, drawing all UI components, and executing our optimized 32-bit memcpy to swap the image to physical video memory in a single instruction sequence.

2. Composite Window Manager (gui.c)

The compositor coordinates individual windows and desktop elements. Key systems include:

- Draggable Window Shells: Tracks mouse click coordinates on title bars. Dragging updates window position relative to the global workspace.
- Focus and Z-Ordering: Clicking inside any window brings it to the top of the render hierarchy, updating the composite stack list.
- Interactive Terminal Window: Incorporates a functional text terminal. System shell commands can be typed and run within the desktop interface.
- Taskbar and Start Menu: A bottom taskbar lists active windows, features interactive menu entries, and renders a live CPU clock.
- About and System Info Dialogs: Renders standard system specifications, physical memory allocations, and custom visual gradients.

3. PS/2 Hardware Drivers (keyboard.c, mouse.c)

Interactive inputs are captured via PS/2 keyboard and mouse drivers. The keyboard driver translates raw hardware make/break scancodes into printable ASCII characters. The mouse driver coordinates standard PS/2 data packets (3-byte streams detailing buttons, X-relative movement, and Y-relative movement). The compositor uses these coordinates to update mouse cursor positions and click collisions on the desktop.

7. Integrated JIT Forth Compiler & Flat FS

In addition to standard shell commands (such as help, mem, gui, uptime, reboot), MyOS features a custom-built, JIT-compiling Forth interpreter (src/compiler.c). This shell allows the user to define functions, execute stack-based algorithms, and compile high-level Forth definitions directly into 32-bit x86 machine instructions on the heap.

1. Stack-Based Forth Interpreter

The Forth engine operates on two private stacks: the Parameter Stack (for arithmetic operations) and the Return Stack (for nested function calls). The compiler parses tokens and resolves them against a dictionary. Standard words include standard stack operations (dup, drop, swap, over) and math operations (+, -, *, /).

2. Native x86 JIT Assembly Compiler

When in compilation mode (triggered by the colon word ':'), the compiler compiles statements. Rather than interpreting bytecode at runtime, the compiler writes native x86 machine code bytes directly into a page-aligned heap memory buffer. When a custom word is called, the CPU jumps directly to the heap buffer address, executing the custom routine at full native bare-metal speed!

```
// Conceptual x86 machine bytes emitted by Forth JIT compiler:
// Word Definition: : ADD_FIVE 5 + ;
// Standard JIT machine instructions generated on the heap:
8B 06          // mov eax, [esi]       ; Load top stack item
05 05 00 00 00 // add eax, 5           ; Add five literal
89 06          // mov [esi], eax       ; Put back to top stack
C3             // ret                  ; Return to main loop
```

3. Custom Flat Filesystem (myfs.c)

MyOS implements a direct-sector mapped filesystem called MyFS. Due to its lightweight flat-sector architecture, it avoids the overhead of FAT or Ext tables. It stores simple files, graphical screens, and Forth scripts directly on hard drive sectors. The terminal shell can list mounted directory contents (myfs_list) and execute scripts containing Forth formulas dynamically using simple sector buffers.

8. VirtualBox Deployment & Implementation Details

Running MyOS under Oracle VirtualBox using our optimized custom legacy BIOS bootloader requires converting the raw disk image into VirtualBox's native VDI format, and ensuring the media registry aligns with the exact UUID expected by the virtual machine configuration.

Step-by-Step Compilation & Deployment

- Compile under WSL / Linux: In your terminal, run the build script: 'wsl bash ./build_linux.sh'. This builds all ASM boot sectors, links the core kernel binary, pads it to 20MB, and writes the output raw image to 'myos_disk.img'.
- Clean Existing VirtualBox Mediums: If an older 'myos_disk.vdi' exists in the workspace, delete it to prevent VirtualBox registration collisions.
- Convert to VDI Format: Run VBoxManage to convert the raw image: 'VBoxManage convertfromraw myos_disk.img myos_disk.vdi --format VDI'.
- Synchronize Media UUID: Set the exact disk UUID expected by the VM media registry to prevent E_FAIL errors: 'VBoxManage internalcommands sethduuid myos_disk.vdi 9b206163-e2bb-4300-9c55-0a11d8bb0122'.
- Attach to VM: In VirtualBox settings, attach the 'myos_disk.vdi' as the Primary Master under the IDE controller.
- Power On: Start the VM. The custom BIOS bootloader stack will initialize the graphical console instantly, displaying full boot diagnostics.

QUICK DEPLOYMENT SCRIPT (BUILD_VDI.PS1)

Write and run this quick PowerShell command sequence inside the myos directory:

```
wsl bash ./build_linux.sh
if (Test-Path myos_disk.vdi) { Remove-Item -Force myos_disk.vdi }
& 'C:\Program Files\Oracle\VirtualBox\VBoxManage.exe' convertfromraw myos_disk.img myos_disk.vdi --format VDI
& 'C:\Program Files\Oracle\VirtualBox\VBoxManage.exe' internalcommands sethduuid myos_disk.vdi 9b206163-e2bb-4300-9c55-0a11d8bb0122
```

Summary of Project Files

MyOS source layout consists of: boot1.asm & boot2.asm (Real-mode custom bootloaders), boot.asm & linker.ld (Multiboot compatibility files), kernel.c & kernel.h (System entry & definitions), string.c (Fast memory loops), gdt.c & idt.c (System gates), timer.c (PIT interrupts), pmm.c (Bitmap memory), framebuffer.c & font.c (Graphics layout), gui.c (Composite window manager), keyboard.c & mouse.c (PS/2 drivers), ata.c (Storage PIO driver), myfs.c (Flat storage filesystem), and compiler.c (Forth JIT engine).